

Práctica: Creación y Gestión de Base de Datos EcommerceDB

1. Configuración Inicial

Create a new database

```
SQLQuery2.s...DB (sa (80))* Novedades
1 CREATE DATABASE EcommerceDB;
2 GO
3
4 USE EcommerceDB;
5 GO
```

2. Creación de Tablas y Carga de Datos

Create core tables with constraints

```
SQLQuery2.s...DB (sa (80))* Novedades
1 USE EcommerceDB;
2 GO
3
4 -- Create Supplier table
5 CREATE TABLE Supplier (
6     SupplierID INT PRIMARY KEY IDENTITY(1,1),
7     SupplierName NVARCHAR(100) NOT NULL UNIQUE,
8     Country NVARCHAR(50) NOT NULL,
9     Email NVARCHAR(100),
10    Phone NVARCHAR(20),
11    CreatedDate DATETIME2 DEFAULT GETUTCDATE()
12 );
13
14 -- Create Category table
15 CREATE TABLE Category (
16     CategoryID INT PRIMARY KEY IDENTITY(1,1),
17     CategoryName NVARCHAR(100) NOT NULL UNIQUE,
18     Description NVARCHAR(500)
19 );
```

100 % No se encontraron problemas.

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-21T13:33:44.9642727+02:00

Insert sample data into the tables

```

SQLQuery2.s...DB (sa (80))* Novedades
1  USE EcommerceDB;
2  GO
3
4  -- Insert sample suppliers
5  INSERT INTO Supplier (SupplierName, Country, Email, Phone)
6  VALUES
7      ('Contoso Supplies', 'USA', 'contact@contoso.com', '555-0100'),
8      ('Fabrikam Inc', 'Canada', 'sales@fabrikam.com', '555-0200');
9
10 -- Insert sample categories
11 INSERT INTO Category (CategoryName, Description)
12 VALUES
13     ('Electronics', 'Electronic devices and accessories'),
14     ('Clothing', 'Apparel and fashion items');
15
16 -- Insert sample products
17 INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
18 VALUES
19     ('Laptop', 1, 1, 1200, 10),
20     ('Smartphone', 1, 1, 800, 20),
21     ('T-Shirt', 2, 2, 20, 50),
22     ('Jeans', 2, 2, 30, 30),
23     ('Headphones', 1, 1, 150, 15),
24     ('Smartwatch', 1, 1, 250, 10),
25     ('Sneakers', 2, 2, 60, 25),
26     ('Dress', 2, 2, 40, 15),
27     ('Backpack', 1, 1, 50, 20),
28     ('Wallet', 1, 1, 30, 30);
29
30 Mensajes
31
32 (2 filas afectadas)
33
34 (2 filas afectadas)
35
36 (2 filas afectadas)
37
38 Hora de finalización: 2026-09-21T13:34:13.7034889+02:00

```

Configuración adicional de la base de datos

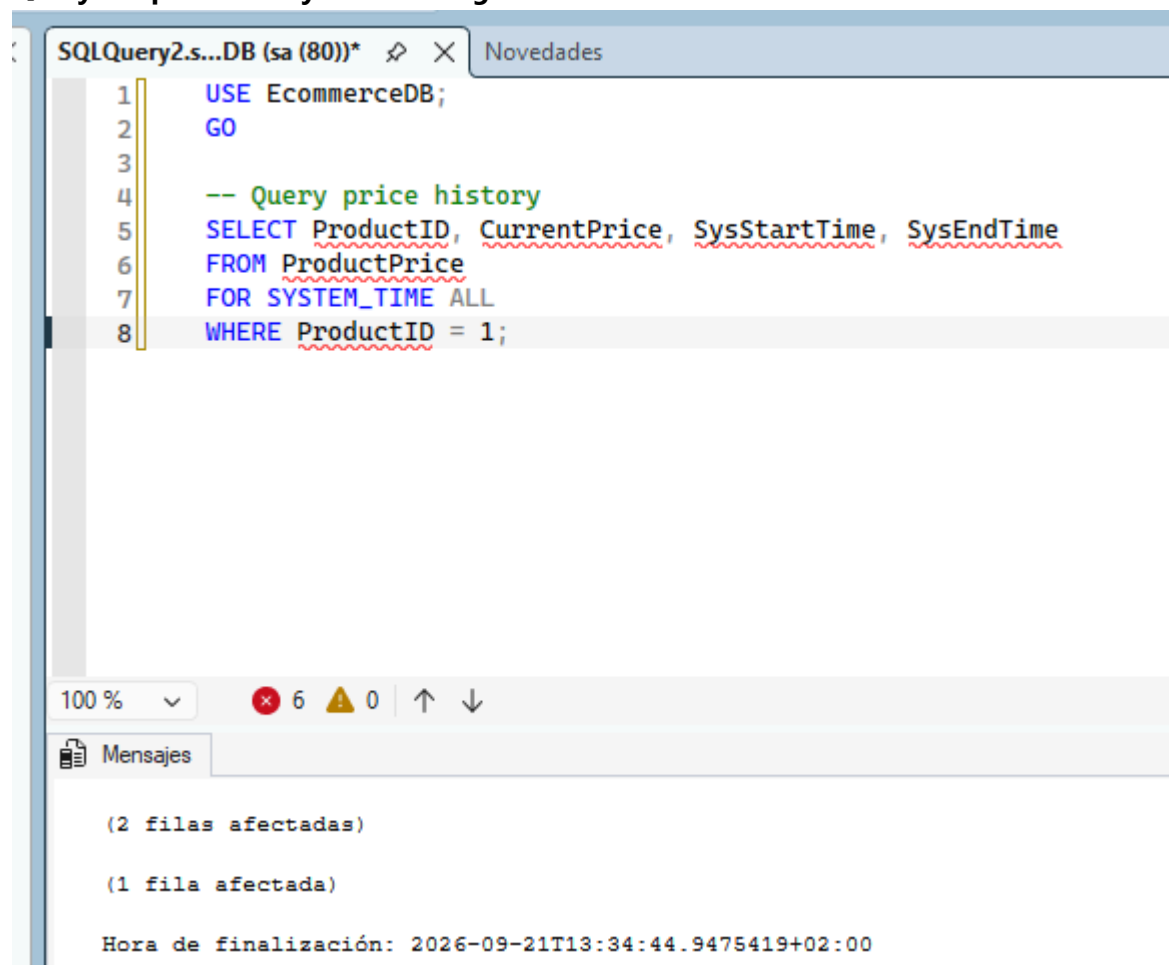
```

SQLQuery2.s...DB (sa (80))* Novedades
1  USE EcommerceDB;
2  GO
3
4  -- Create Price History table with temporal versioning
5  CREATE TABLE ProductPrice (
6      PriceID INT PRIMARY KEY IDENTITY(1,1),
7      ProductID INT NOT NULL,
8      CurrentPrice DECIMAL(10,2) NOT NULL,
9      EffectiveDate DATE,
10     SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START HIDDEN,
11     SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END HIDDEN,
12     PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime),
13     FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
14 ) WITH (SYSTEM_VERSIONING = ON);
15  GO
16
17 -- Insert initial price data
18 INSERT INTO ProductPrice (ProductID, CurrentPrice, EffectiveDate)
19 VALUES
20     (1, 1200, '2026-09-21'),
21     (2, 800, '2026-09-21'),
22     (3, 20, '2026-09-21'),
23     (4, 30, '2026-09-21'),
24     (5, 150, '2026-09-21'),
25     (6, 250, '2026-09-21'),
26     (7, 60, '2026-09-21'),
27     (8, 40, '2026-09-21'),
28     (9, 50, '2026-09-21'),
29     (10, 30, '2026-09-21');
30
31 Mensajes
32
33 (2 filas afectadas)
34
35 (1 fila afectada)
36
37 Hora de finalización: 2026-09-21T13:34:44.9475419+02:00

```

3. Consultas de Historial

Query the price history to see changes over time



```
SQLQuery2.s...DB (sa (80))* Novedades
1  USE EcommerceDB;
2  GO
3
4  -- Query price history
5  SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
6  FROM ProductPrice
7  FOR SYSTEM_TIME ALL
8  WHERE ProductID = 1;
```

100 % 6 0 ↑ ↓

Mensajes

(2 filas afectadas)

(1 fila afectada)

Hora de finalización: 2026-09-21T13:34:44.9475419+02:00

4. Gestión de Datos JSON

Implementación de metadatos JSON

SQLQuery2.s...DB (sa (80))* Novedades

```
1 USE EcommerceDB;
2 GO
3
4 -- Add metadata column to Product (JSON type requires SQL Server 2025)
5 ALTER TABLE Product ADD Metadata JSON;
6 GO
7
8 -- Add computed column for indexing
9 ALTER TABLE Product ADD MetadataColor AS JSON_VALUE(Metadata, '$.color');
10 GO
11
12 -- Create index on the computed column
13 CREATE NONCLUSTERED INDEX IX_Product_Metadata_Color
14 ON Product (MetadataColor);
15 GO
16
17 -- Update products with metadata
18 UPDATE Product SET Metadata = N'{"color": "blue", "size": "large", "material": "cotton"}'
```

100 % 8 0

Mensajes

(2 filas afectadas)

(1 fila afectada)

Hora de finalización: 2026-09-21T13:34:44.9475419+02:00

Query the JSON data

SQLQuery2.s...DB (sa (80))* Novedades

```
1 USE EcommerceDB;
2 GO
3
4 -- Query JSON data
5 SELECT
6     ProductID,
7     ProductName,
8     JSON_VALUE(Metadata, '$.color') AS Color,
9     JSON_VALUE(Metadata, '$.size') AS Size,
10    JSON_VALUE(Metadata, '$.material') AS Material
11 FROM Product
12 WHERE JSON_VALUE(Metadata, '$.color') = 'blue';
```

100 % 7 0

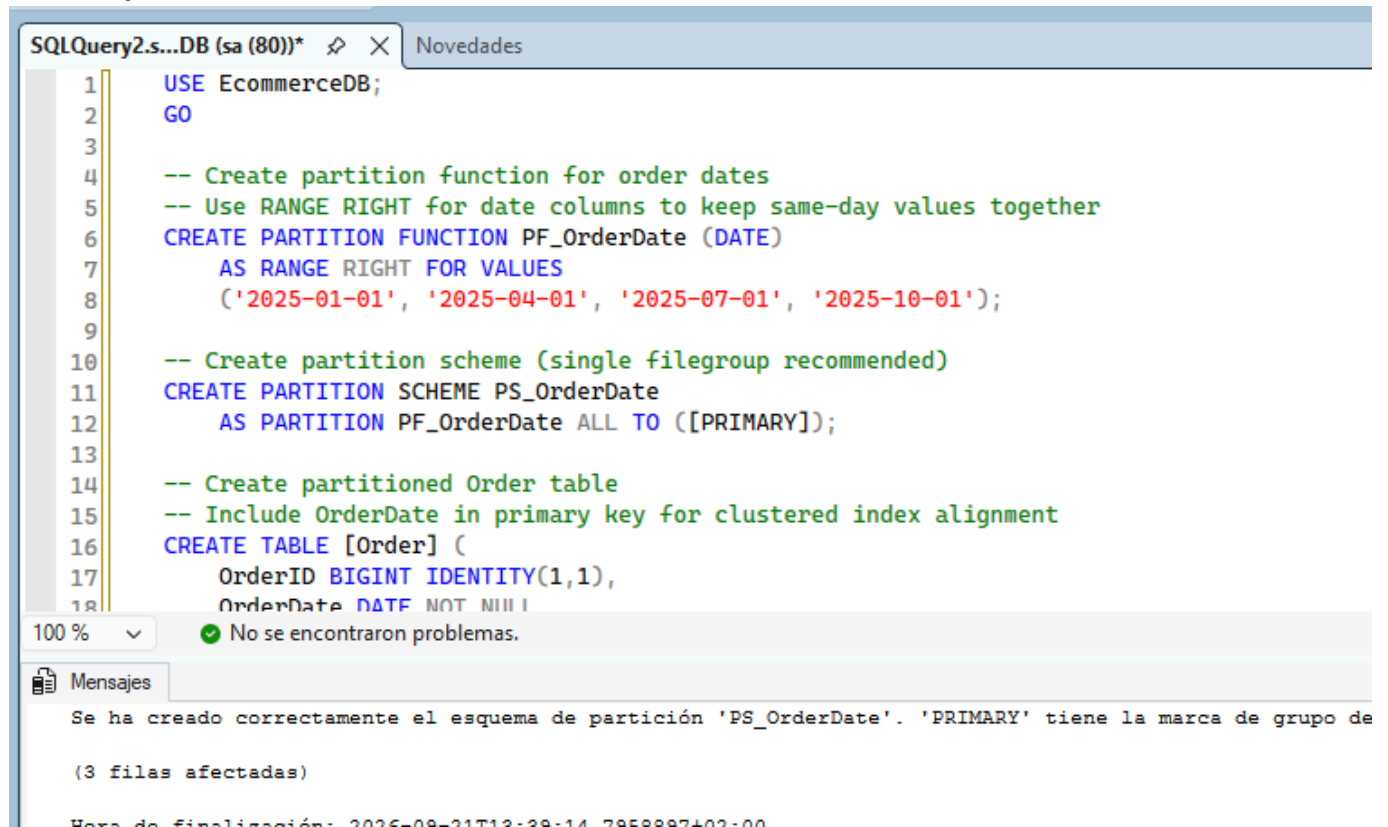
Resultados Mensajes

	ProductID	ProductName	Color	Size	Material
1	1	Wireless Mouse	blue	large	cotton

5. Particionamiento de Tablas

Partitioning divides large tables into smaller segments for faster queries and easier maintenance. This task creates a partitioned orders table.

Create a partitioned order table



The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays a SQL query window titled 'SQLQuery2.s...DB (sa (80))*' with a 'Novedades' tab. The query is as follows:

```
1  USE EcommerceDB;
2  GO
3
4  -- Create partition function for order dates
5  -- Use RANGE RIGHT for date columns to keep same-day values together
6  CREATE PARTITION FUNCTION PF_OrderDate (DATE)
7  AS RANGE RIGHT FOR VALUES
8  ('2025-01-01', '2025-04-01', '2025-07-01', '2025-10-01');
9
10 -- Create partition scheme (single filegroup recommended)
11 CREATE PARTITION SCHEME PS_OrderDate
12 AS PARTITION PF_OrderDate ALL TO ([PRIMARY]);
13
14 -- Create partitioned Order table
15 -- Include OrderDate in primary key for clustered index alignment
16 CREATE TABLE [Order] (
17     OrderID BIGINT IDENTITY(1,1),
18     OrderDate DATE NOT NULL
```

The bottom pane shows the 'Mensajes' (Messages) window. It displays a green checkmark icon and the text: 'No se encontraron problemas.' (No problems were found). Below this, a message states: 'Se ha creado correctamente el esquema de partición 'PS_OrderDate'. 'PRIMARY' tiene la marca de grupo de (3 filas afectadas)' (The partition scheme 'PS_OrderDate' has been created successfully. 'PRIMARY' has the group mark (3 rows affected)). At the bottom, the completion time is shown: 'Hora de finalización: 2026-09-21T12:38:14.7858887403+00'.

Query by partition

SQLQuery2.s...DB (sa (80))* Novedades

```
1  USE EcommerceDB;
2  GO
3
4  -- Query by partition
5  SELECT
6      $PARTITION.PF_OrderDate(OrderDate) AS PartitionNumber,
7      COUNT(*) AS OrdersInPartition,
8      MIN(OrderDate) AS MinDate,
9      MAX(OrderDate) AS MaxDate
10 FROM [Order]
11 GROUP BY $PARTITION.PF_OrderDate(OrderDate);
```

100 % 1 0

Resultados Mensajes

	PartitionNumber	OrdersInPartition	MinDate	MaxDate
1	2	2	2025-01-15	2025-02-20
2	3	1	2025-06-10	2025-06-10

6. Uso de Secuencias

Sequences generate unique numbers independently of any table. This task uses a sequence for order line item identifiers.

Create order details with SEQUENCE

SQLQuery2.s...DB (sa (80))* Novedades

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18

USE EcommerceDB;
GO

-- Create SEQUENCE for order line items
CREATE SEQUENCE OrderLineSequence
START WITH 1
INCREMENT BY 1;

-- Create OrderDetail table
CREATE TABLE OrderDetail (
OrderLineID INT PRIMARY KEY,
OrderID BIGINT NOT NULL,
OrderDate DATE NOT NULL,
ProductID INT NOT NULL,
Quantity INT NOT NULL,
UnitPrice DECIMAL(10,2) NOT NULL,
LineTotal AS (Quantity * UnitPrice),
CHECK (Quantity > 0))

100 % 2 0

Mensajes

(3 filas afectadas)

Hora de finalización: 2026-09-21T13:40:09.7556407+02:00

SQLQuery2.s...DB (sa (80))* Novedades

1
2
3
4

USE EcommerceDB;
GO

SELECT * FROM OrderDetail;

100 % 1 0

Resultados Mensajes

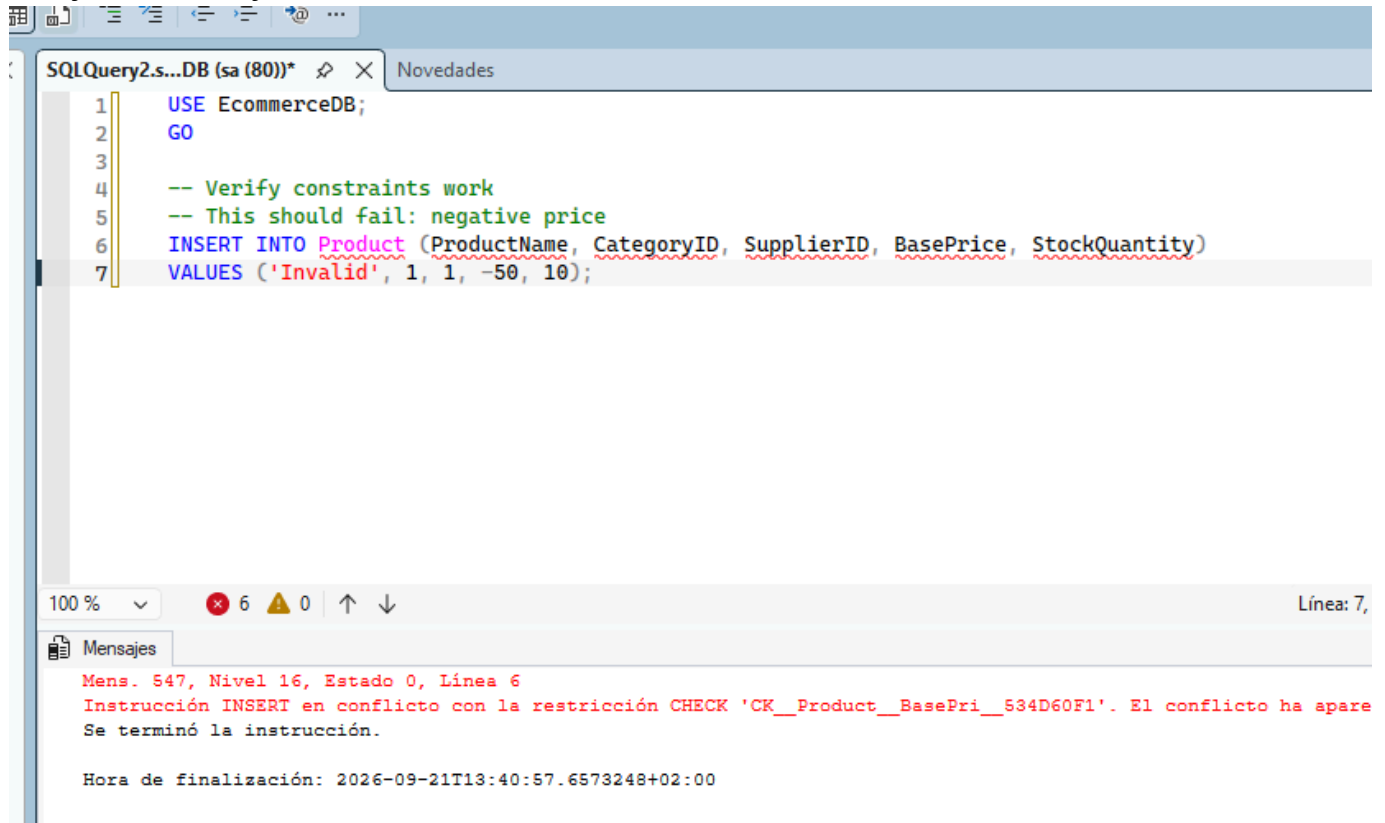
	OrderLineID	OrderID	OrderDate	ProductID	Quantity	UnitPrice	LineTotal
1	1	1	2025-01-15	1	2	99.99	199.98
2	2	1	2025-01-15	2	1	149.99	149.99
3	3	2	2025-02-20	1	3	99.99	299.97

Verify the data

7. Verificación de Objetos y Restricciones

Run verification queries to ensure all database objects were created correctly.

Verify database objects



```
SQLQuery2.s...DB (sa (80))* Novedades
1  USE EcommerceDB;
2  GO
3
4  -- Verify constraints work
5  -- This should fail: negative price
6  INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
7  VALUES ('Invalid', 1, 1, -50, 10);
```

100 % 6 0 Línea: 7,

Mensajes

Mens. 547, Nivel 16, Estado 0, Línea 6
Instrucción INSERT en conflicto con la restricción CHECK 'CK_Product_BasePri_534D60F1'. El conflicto ha apareado
Se terminó la instrucción.

Hora de finalización: 2026-09-21T13:40:57.6573248+02:00

 **Nota de validación:** This query should fail with a CHECK constraint violation, confirming that the constraint is working correctly.

Verify the JSON and partitioning queries

SQLQuery2.s...DB (sa (80))*

Novedades

```
1  USE EcommerceDB;
2  GO
3
4  -- Verify JSON queries work
5  SELECT ProductName, JSON_VALUE(Metadata, '$.color') AS Color
6  FROM Product
7  WHERE Metadata IS NOT NULL;
8
9  -- Verify partitioning
10 SELECT $PARTITION.PF_OrderDate(OrderDate) AS Partition, COUNT(*) AS RecordCount
11 FROM [Order]
12 GROUP BY $PARTITION.PF_OrderDate(OrderDate);
13
14 -- Verify temporal table
15 SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
16 FROM ProductPrice FOR SYSTEM_TIME ALL
17 ORDER BY ProductID, SysStartTime;
```

100 % 12 0

Resultados Mensajes

	ProductName	Color
1	Wireless Mouse	blue
2	Cotton T-Shirt	red

	Partition	RecordCount
1	2	2
2	3	1

	ProductID	CurrentPrice	SysStart Time	SysEndTime
1	1	99.99	2026-09-21 11:34:44.9395357	2026-09-21 11:34:44.9405269
2	1	109.99	2026-09-21 11:34:44.9405269	9999-12-31 23:59:59.9999999
3	2	149.99	2026-09-21 11:34:44.9395357	9999-12-31 23:59:59.9999999